Funcionamento dos tokens

Perambulando | Atualização: 18/10/2026

Base: `Domain/Service/JwtTokenService.cs`, `Presentation/Program.cs`, controllers de User/Host e fluxo de sessão do protótipo.

1. Fluxo atual

```
Login com e-mail/senha
  -> Identity verifica usuário e senha
  -> JwtTokenService consulta roles e emite JWT
  -> frontend guarda accessToken
  -> Authorization: Bearer TOKEN nas chamadas protegidas
  -> middleware valida o token
  -> controller lê o usuário e serviço verifica propriedade
```

O endpoint de login retorna `accessToken`, `tokenType` (Bearer) e `expiresAt`. O cadastro de host e a edição de usuário também emitem tokens, dentro de `auth` na resposta.

2. O que existe no JWT

Claim	Significado
`sub`	ID do usuário; usado pelo UserManager para identificar o solicitante
`jti`	Identificador novo e aleatório para cada token emitido
`name`	`UserName` no momento da emissão
`isPro`	Valor da propriedade de assinatura/condição Pro do usuário
`typeofuser`	Enum numérico do tipo Local/Turist
`role`	Uma claim por role cadastrada no Identity
`iss`, `aud`	Emissor e público-alvo configurados
`nbf`, `exp`	Instante inicial de validade e expiração

O token é assinado com HMAC SHA-256. A assinatura permite verificar integridade; o conteúdo não é criptografado. Nenhuma senha é colocada no JWT. O token não leva a lista de hosts nem concede propriedade de um host por si só.

3. Configuração exigida

Configuração	Variável de ambiente equivalente	Regra
`Jwt:Key`	`Jwt__Key`	Mínimo de 32 bytes UTF-8; chave privada da aplicação
`Jwt:Issuer`	`Jwt__Issuer`	Não vazio
`Jwt:Audience`	`Jwt__Audience`	Não vazio
`Jwt:ExpirationHours`	`Jwt__ExpirationHours`	Inteiro positivo

O startup rejeita chave ausente/curta, emissor vazio, audiência vazia ou expiração não positiva. Não são reproduzidos valores de credenciais neste documento.

4. Validação de cada chamada

O JwtBearer valida assinatura, emissor, audiência e tempo de validade. A tolerância de relógio configurada é de 30 segundos. `MapInboundClaims = false` preserva os nomes das claims; as configurações do Identity usam `sub`, `name` e `role`.

```
var userId = _userManager.GetUserId(User);
```

O controller obtém o ID do token validado. Não deve usar um userId escolhido pelo frontend como prova de identidade. `[Authorize]` exige autenticação; `[Authorize(Roles = "ADMIN,HOST")]` exige pelo menos uma dessas roles; `[AllowAnonymous]` permite consulta sem login.

Autenticação responde quem é a pessoa. A autorização por role permite acessar determinada funcionalidade. A verificação `Hosts.UserId == userId` nos serviços determina a propriedade efetiva.

5. Atualização após criar host

A conta pode iniciar sem role HOST. O cadastro de host recebe o JWT existente, associa o host ao usuário e atribui HOST se necessário. Depois gera um novo token que já contém as roles atuais.

O frontend deve substituir o token local por `createHost.auth.accessToken`. O token anterior não passa a ganhar novas claims automaticamente. Não é necessário repetir o login se o novo token já foi aplicado.

O mesmo princípio vale para `PUT /api/User/Edit`: aplicar `auth.accessToken` para refletir nome e demais claims atualizadas.

6. Expiração e logout

Quando o JWT expira, chamadas protegidas deixam de ser autenticadas e normalmente recebem 401, considerando a tolerância de relógio. No fluxo atual, o usuário faz login novamente para obter outro token. Não há refresh token nem endpoint de renovação.

Logout é local: o frontend remove o token e limpa os dados de sessão. O protótipo tem o botão Limpar sessão para isso. Não há endpoint de logout, lista de revogação nem consulta de revogação por `jti`.

Uma cópia do token permanece tecnicamente válida até expirar. Emitir outro token não invalida os anteriores. Um token removido do armazenamento local não foi revogado no servidor.

7. Limites atuais

- O middleware não consulta a situação do usuário e suas roles no banco a cada token validado. Alguns fluxos verificam usuário ativo explicitamente; as verificações não são universais para todas as actions.
- Alterar/remover uma role no banco não retira a claim de um token já emitido.
- `jti` existe, mas não é persistido ou usado como mecanismo de revogação.
- A existência de `AspNetUserTokens` no Identity não significa que os access tokens da aplicação estejam sendo gravados ali.
- A atribuição de HOST e a criação do host não estão em uma transação única no fluxo atual.

8. Contrato para o frontend

Enviar o Bearer apenas para a API apropriada, aplicar os novos tokens retornados por create de host e edit de usuário, tratar 401 como necessidade de autenticação e distinguir 403 (role insuficiente) de 404 (registro indisponível para o solicitante).

`canEdit` e `canDelete` não são retornados atualmente. A interface pode usar a listagem dos próprios hosts para gestão; o backend deve continuar verificando o dono em toda mutação.